
Graph-Based Skill Discovery in Continuous State Spaces via Temporal Distance Clustering

Harvey Ayling Özgür Şimşek Joshua B. Evans
Department of Computer Science
University of Bath, Bath, United Kingdom
{hja43|o.simsek|jbe25}@bath.ac.uk

Abstract

To solve complex problems with long horizons, intelligent agents must create useful temporally-extended actions, also known as *skills*. Graph-based skill discovery methods identify useful skills by using the environment’s connectivity structure when modelled as a graph. This approach assumes a discrete state space. How best to construct graphs for environments with continuous state spaces remains an open research problem. We present an approach for constructing abstract transition graphs by clustering states within a temporal distance embedding space. This approach enables the application of any graph-based skill discovery method to environments with continuous state spaces. We empirically show that the proposed approach successfully captures underlying environment connectivity and supports the application of diverse skill discovery algorithms.

1 Introduction

To operate effectively in complex environments, intelligent agents need to act, learn, and plan across multiple timescales. Useful, reusable temporally-extended actions are known as *skills*. One prominent set of approaches to discovering skills models an environment’s transition structure as a graph, and uses graph-theoretic algorithms, such as vertex centrality [1–4], clustering [5–10], or spectral [11–13] algorithms, to create skills based on the graph’s connectivity structure.

Modelling an environment’s transition structure as a graph is natural for environments with finite, discrete state spaces; each state can be represented as a distinct vertex, and transitions as weighted directed edges. However, extending such graphical representations to continuous state spaces is not straightforward. Naive discretisation, such as clustering states using the Euclidean distance between raw state representations, fails because Euclidean distance rarely reflects an environment’s connectivity: states separated by thin barriers or irreversible transitions may be geometrically close yet temporally distant.

While recent Laplacian-based methods [14–17] bypass explicit graph construction by approximating eigenvectors directly via function approximation, it remains unclear how best to adapt clustering and centrality-based skill discovery methods to domains with continuous state spaces, since they inherently rely on explicit graph structures. Consequently, the benefits of these approaches, such as identifying bottleneck subgoals and constructing multi-level skill hierarchies, remain inaccessible in continuous-state environments.

We propose an approach for constructing abstract graphical representations by clustering states within a learned temporal distance embedding space. We evaluate the proposed approach across three continuous-state environments and show that the generated graphs accurately recover structural features. Furthermore, we show that this approach is versatile, enabling the application of three representative graph-based skill discovery algorithms: betweenness skills [1], the Louvain skill hierarchy [5], and eigenoptions [11].

2 Background

We model the interaction between an agent and environment as a Markov decision process (MDP) [18]. We denote $\Delta(\mathcal{X})$ as the set of probability distributions over a set $\mathcal{X}$. An MDP is defined as a six-tuple $(\mathcal{S}, \mathcal{A}, R, P, d_0, \gamma)$, where $\mathcal{S}$ is a set of states, $\mathcal{A}$ is a set of actions, $P : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the transition dynamics, $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the reward function, $d_0 \in \Delta(\mathcal{S})$ is the initial state distribution and $\gamma \in [0, 1]$ is the discount factor. At step $t \in \{0, 1, 2, \dots\}$, the agent observes state S_t , chooses action A_t , and consequently transitions to state S_{t+1} and receives reward R_{t+1} . The agent’s objective is to learn a policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ to maximise a return, for instance, the sum of discounted future rewards, $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

Options. The actions of an MDP take exactly one step to execute; these are called *primitive actions*. Using primitive actions, one can define *abstract actions*, whose execution can take a variable number of steps, also called *skills*. We represent skills via the options framework [19, 20]. An option o is a triple $(\mathcal{I}_o, \pi_o, \beta_o)$, where $\mathcal{I}_o \subseteq \mathcal{S}$ is the initiation set, $\pi_o : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ is the option policy, and $\beta_o : \mathcal{S} \rightarrow [0, 1]$ is the termination condition. An option is available in state s if $s \in \mathcal{I}_o$ and, when executed, the agent follows the option policy π_o . At each subsequent transition into a state s' , the option terminates with probability $\beta_o(s')$ and continues execution with probability $1 - \beta_o(s')$.

Graphs. A directed graph G is a pair $(\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of vertices and $\mathcal{E}$ is a set of edges which are ordered pairs of connected vertices. A weighted graph assigns a real number to each edge, $w : \mathcal{E} \rightarrow \mathbb{R}$. For a graph with n vertices, $\mathcal{V} = \{v_1, \dots, v_n\}$, $W \in \mathbb{R}^{n \times n}$ denotes the weight matrix, where each entry $W_{ij} = w(v_i, v_j)$ is the edge weight between vertices v_i and v_j , or 0 if no edge exists. The degree matrix $D \in \mathbb{R}^{n \times n}$ is the diagonal matrix with entries $D_{ii} = \sum_j W_{ij}$, the sum of the edge weights connected to vertex v_i .

For an MDP with a finite state space, the state transition graph is a weighted directed graph $G = (\mathcal{S}, \mathcal{E}, w)$, where the vertices are the set of environment states, and edges correspond to possible transitions between states, $\mathcal{E} = \{(s, s') \in \mathcal{S}^2 \mid \exists a : P(s' \mid s, a) > 0\}$. Various edge-weighting schemes can be used, such as unweighted [5, 6], proportional to how frequently the transition is experienced [21], proportional to the underlying probability of transition [22], or proportional to the expected reward [1].

Graph-based skill discovery. Graph-based skill discovery methods create options from the connectivity structure of the state transition graph. Betweenness centrality [23, 24] is a vertex metric that quantifies the fraction of all-pairs shortest paths passing through a given vertex: $f_{bc}(s) = \sum_{u \neq s \neq v} \sigma_{uv}(s) / \sigma_{uv}$, where σ_{uv} is the total number of shortest paths from vertex u to vertex v and $\sigma_{uv}(s)$ is the number of those paths that pass through vertex s . Because many shortest paths naturally funnel through the sparse connections between densely connected regions, local maxima of betweenness identify topological bottlenecks, such as doorways. In betweenness skills [1], these bottleneck states are selected as subgoals, and options are trained to navigate to them, enabling the agent to transition efficiently between distinct regions of the environment.

The Louvain algorithm [25] partitions a graph into communities $\{c_1, c_2, \dots, c_k\}$ by maximising modularity, defined as $\sum_{i=1}^k (e_{ii} - \eta a_i^2)$, where e_{ii} is the proportion of total edge weight that connects vertices within cluster c_i , a_i is the fraction of total edge weight incident to vertices in cluster c_i , and η is a resolution parameter. High-modularity partitions isolate densely connected regions separated by sparse boundaries, naturally reflecting the environment’s community structure. Because the Louvain algorithm operates recursively by aggregating clusters together, it discovers partitions at multiple levels of granularity. The Louvain skill hierarchy [5] uses this to create a hierarchy of options for navigating between adjacent clusters at every level, enabling efficient movement across multiple levels of abstraction.

Given the degree matrix D and weight matrix W , the normalised graph Laplacian is defined as $N = D^{-1/2}(D - W)D^{-1/2}$. Computing its eigendecomposition yields orthonormal eigenvectors ordered by ascending eigenvalue. The eigenvectors associated with the smallest non-zero eigenvalues correspond to the slowest diffusion modes of a random walk, capturing the environment’s global geometry and large-scale axes of variation [26]. In eigenoptions [11], options are created to drive an agent along the gradients of these eigenvectors, enabling efficient directed exploration.

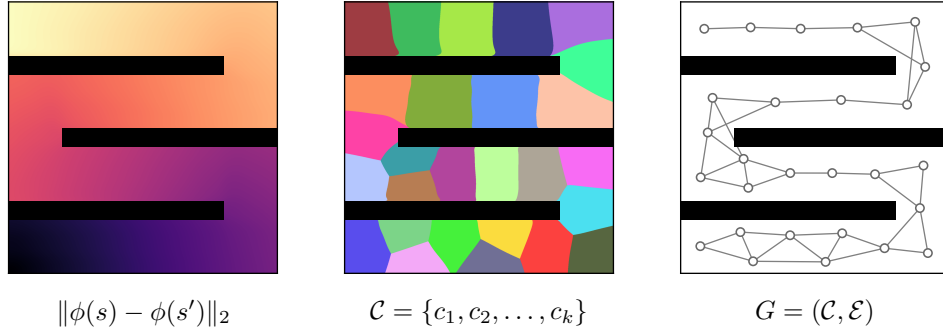


Figure 1: States are embedded in a temporal distance space, then clustered, enabling construction of an abstract transition graph.

3 Related Work

Hierarchical reinforcement learning is a broad research area that concerns agents learning, planning, and acting across multiple timescales. Graph-based methods identify useful temporally extended behaviours by representing an environment as a graph and applying methods such as clustering, centrality, or spectral algorithms [27].

Centrality-based approaches [1–4] assign scalar values to states to quantify some notion of their structural importance on the graph. A common approach is to identify subgoals as local maxima of the chosen centrality metric, and then define skills to efficiently navigate the agent to these subgoals. Clustering-based approaches [5–10] partition the state space and define skills to navigate efficiently between adjacent clusters. Spectral approaches [11–13] compute the eigenvectors of the normalised graph Laplacian to identify principal directions of the state space; skills that follow these directions allow efficient traversal of the environment over varying timescales.

While spectral methods can scale to large, continuous-state environments by approximating the eigenvectors without explicitly building a state transition graph [14–17], centrality and clustering approaches still fundamentally rely on explicit graph structures. Given an appropriate graphical representation, there is no conceptual obstacle to applying these methods to continuous domains; bridging this gap is the primary motivation for this work.

Abstractions in continuous spaces. Several approaches have addressed this challenge by extending discrete abstractions to continuous domains. Generative models, such as discrete variational autoencoders paired with the successor representation [28] or vector-quantised variational autoencoders [29], have been used to discretise state spaces for skill discovery and curriculum learning. However, a shared limitation of these representation-learning approaches is the requirement to pre-specify a maximum number of abstract states, limiting flexibility across environments of varying complexity. Other works have explored clustering via Dirichlet process mixture models [30], or constructing graphs implicitly from the embeddings of a next-frame prediction model for Laplacian-based skills [12, 31]. Additionally, Lee et al. [32] construct ‘landmark’ graphs for planning using temporal distances. While effective in specific contexts, these methods often require significant computational overhead or lack a unified, model-agnostic abstraction approach that supports arbitrary graph-based skill discovery methods.

Clustering algorithms for reinforcement learning. While various clustering algorithms could partition the state space of an environment, many traditional methods exhibit distinct limitations in a reinforcement learning context. The simplest approaches, such as k -means [33, 34], require specifying the number of clusters in advance, which is problematic because an agent is unlikely to know a priori how many regions an environment should be divided into. Dissimilarity-based alternatives, such as k -medoids [35], can operate directly on arbitrary or pairwise distance functions without vector embeddings; however, they still require a pre-specified cluster count and typically have quadratic

time and space complexity with dataset size. Density-based algorithms like H/DBSCAN [36, 37] and OPTICS [38] are sensitive to the agent’s data-collection policy; because state density reflects exploration bias rather than true topology, these methods risk discarding critical but rarely visited states as noise. Gaussian mixture models [39] allow for a flexible number of clusters and varied cluster shapes but are computationally prohibitive when applied to large, high-dimensional continuous spaces and when fit from many data points. Finally, linkage-based hierarchical clustering [40] partitions a fixed dataset rather than the continuous space itself, making it difficult to dynamically determine out-of-sample cluster membership without recomputing the entire hierarchy. In contrast, the BIRCH algorithm [41] is well-suited for large-scale reinforcement learning. As a tree-based method, BIRCH builds a hierarchical summary of the data that scales linearly with dataset sizes. Instead of requiring a fixed number of clusters, it uses a radius threshold to dynamically determine the number of clusters. Furthermore, its tree-based architecture natively supports incremental, online updates and provides immediate cluster assignment for unseen states without recomputing the underlying hierarchy.

4 Proposed Approach

For the purposes of this work, we assume the agent has access to a pre-collected dataset of transitions, denoted by $\mathcal{D} = \{(s_i, a_i, s'_i)\}_{i=1}^N$. We focus on this offline setting to evaluate the viability of the graph construction approach without the confounding dynamics of online exploration. Using an offline dataset of transitions, we construct an abstract transition graph, formally defined below.

Definition 1 (Abstract transition graph). Given an MDP, an abstract transition graph is a directed graph $G = (\mathcal{C}, \mathcal{E})$, where the vertices $c \in \mathcal{C}$, called abstract states, form a partition of the state space $\mathcal{S}$, and edges represent possible transitions between abstract states, $\mathcal{E} = \{(c, c') \in \mathcal{C}^2 \mid \exists s \in c, \exists a \in \mathcal{A} : P(c' \mid s, a) > 0\}$.

To construct this graphical representation, we require a mechanism to determine whether a pair of states should be grouped into the same abstract state. While a naive approach might rely on Euclidean distance between raw state representations, this metric often fails to reflect the environment’s true connectivity. For instance, consider two states, represented by x and y coordinates, separated by a thin wall: although their spatial coordinates are physically close, the actual number of steps required for an agent to navigate between them might be substantial. Clustering on raw features therefore risks grouping disconnected regions across impassable barriers, as shown in Figure 2.

To reflect the environment’s topology, the distance metric should capture temporal reachability rather than arbitrary feature similarity. We therefore employ a temporal distance metric that approximates the expected steps required to transition between two states under an optimal policy. Specifically, we adopt the Hilbert representation [42], training an encoder $\phi : \mathcal{S} \rightarrow \mathbb{R}^d$ such that the Euclidean distance between embeddings, $\|\phi(s) - \phi(s')\|_2$, approximates this shortest-path temporal distance between s and s' . Embedding states directly into a Euclidean vector space is particularly useful because it allows scalable, centroid-based clustering algorithms, avoiding the pairwise distance matrices required by non-Euclidean or pairwise-only distance estimators.

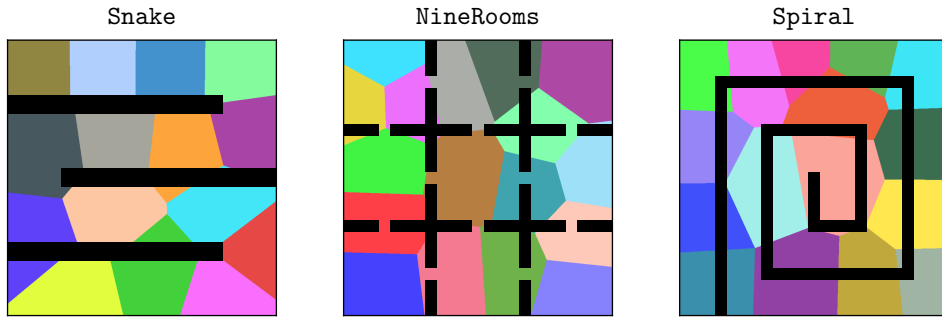


Figure 2: Partitions produced via BIRCH ($\rho = 2.5$) with the Euclidean distance metric between raw state representations.

We embed the states observed in the transition dataset $\mathcal{D}$ into the latent space and partition them using the BIRCH clustering algorithm [41]. BIRCH is well suited for this setting; unlike algorithms such as k -means or k -medoids, it does not require specifying the number of clusters a priori. Instead, BIRCH governs cluster formation via a maximum cluster radius threshold $\rho \in [0, \infty)$. Because distance in the latent space reflects temporal distance, the threshold ρ directly sets the maximum temporal radius of each abstract state. Finally, BIRCH scales linearly with dataset size and constructs a tree-based summary, enabling efficient out-of-sample state assignment without recomputing the partition. Section 3 further discusses the choice of clustering algorithm. Clustering these embedded states yields a set of centroids $\{\mu_1, \mu_2, \dots, \mu_{|\mathcal{C}|}\}$, where each centroid $\mu_c \in \mathbb{R}^d$ defines an abstract state $c \in \mathcal{C}$.

We can define the abstraction mapping $\lambda : \mathcal{S} \rightarrow \mathcal{C}$ by assigning each state to its nearest centroid:

$$\lambda(s) = \arg \min_{c \in \mathcal{C}} \|\phi(s) - \mu_c\|_2.$$

Using this mapping, we construct the abstract transition graph $G = (\mathcal{C}, \mathcal{E})$. A directed edge (c, c') is included in set of edges $\mathcal{E}$ if there exists at least one transition $(s, a, s') \in \mathcal{D}$ such that $\lambda(s) = c$ and $\lambda(s') = c'$. In practice, it may be beneficial to require more observed transitions before creating an edge to filter out low-probability or noisy transitions. Figure 1 illustrates the stages of the proposed approach.

5 Experimental Analysis

Our analysis is motivated by the following questions: Does the proposed approach construct structurally and qualitatively sound abstract transition graphs? Can graph-based skill discovery methods be successfully applied to these graphs? How does the choice of distance threshold affect the structure of the constructed graph?

Environments. The environments, shown in the first row of Figure 3, are Snake, NineRooms, and Spiral. An agent’s state is its 2D position $s = (x, y) \in [-10, 10]^2$, constrained by impassable internal walls and outer boundaries. The agent has four actions: {north, east, south, west}. After taking an action, the agent moves by a distance sampled from $\mathcal{U}(0.5, 1.5)$ in the chosen direction and $\mathcal{U}(-0.1, 0.1)$ perpendicular to it. If an action would move the agent through a boundary or wall, it instead does not move.

Methodology. We collect a dataset of 10^6 transitions from each environment using a uniform random policy. We then train the Hilbert representation [42] on this dataset for 10^6 gradient steps. The encoder is a two-layer network with a hidden width of 256, layer normalisation [43], and GELU [44] activations, optimised with Adam [45] at a learning rate of $\alpha = 0.003$ with a batch size of 1024. We set the latent dimension to $d = 32$, the expectile to $\tau = 0.9$, and the discount factor to $\gamma = 0.99$. Finally, we cluster the embedded states using BIRCH [41] with radius threshold $\rho = 2.5$.

Skills. To evaluate the efficacy of the proposed approach, we consider three representative graph-based skill discovery methods across centrality, clustering, and spectral approaches: betweenness skills [1, 2], the Louvain skill hierarchy [5, 7], and eigenoptions [11].

For betweenness skills [1, 2], because betweenness centrality is computed using graph paths, we assign edge weights using the temporal distance between cluster centroids: $w(u, v) = \|\mu_u - \mu_v\|_2$ for $(u, v) \in \mathcal{E}$, so shortest paths correspond to minimum expected transition times. Figure 3 shows the centroids of the abstract states identified as subgoals.

For the Louvain skill hierarchy [5, 7], the Louvain algorithm interprets edge weights as connection strengths; therefore, we define edge weights as $w(u, v) = \exp(-\|\mu_u - \mu_v\|_2)$ for $(u, v) \in \mathcal{E}$. As the distance between two centroids increases, the connection between them decreases exponentially. We use a modularity resolution parameter of $\eta = 0.05$. The algorithm generated two-level hierarchies for each environment. Figure 3 shows the top level of the generated hierarchies; Figure 5 shows both levels generated for each environment.

For eigenoptions [11], we compute the eigendecomposition of the normalised graph Laplacian. Because the environments feature reversible dynamics, transitions between adjacent abstract states

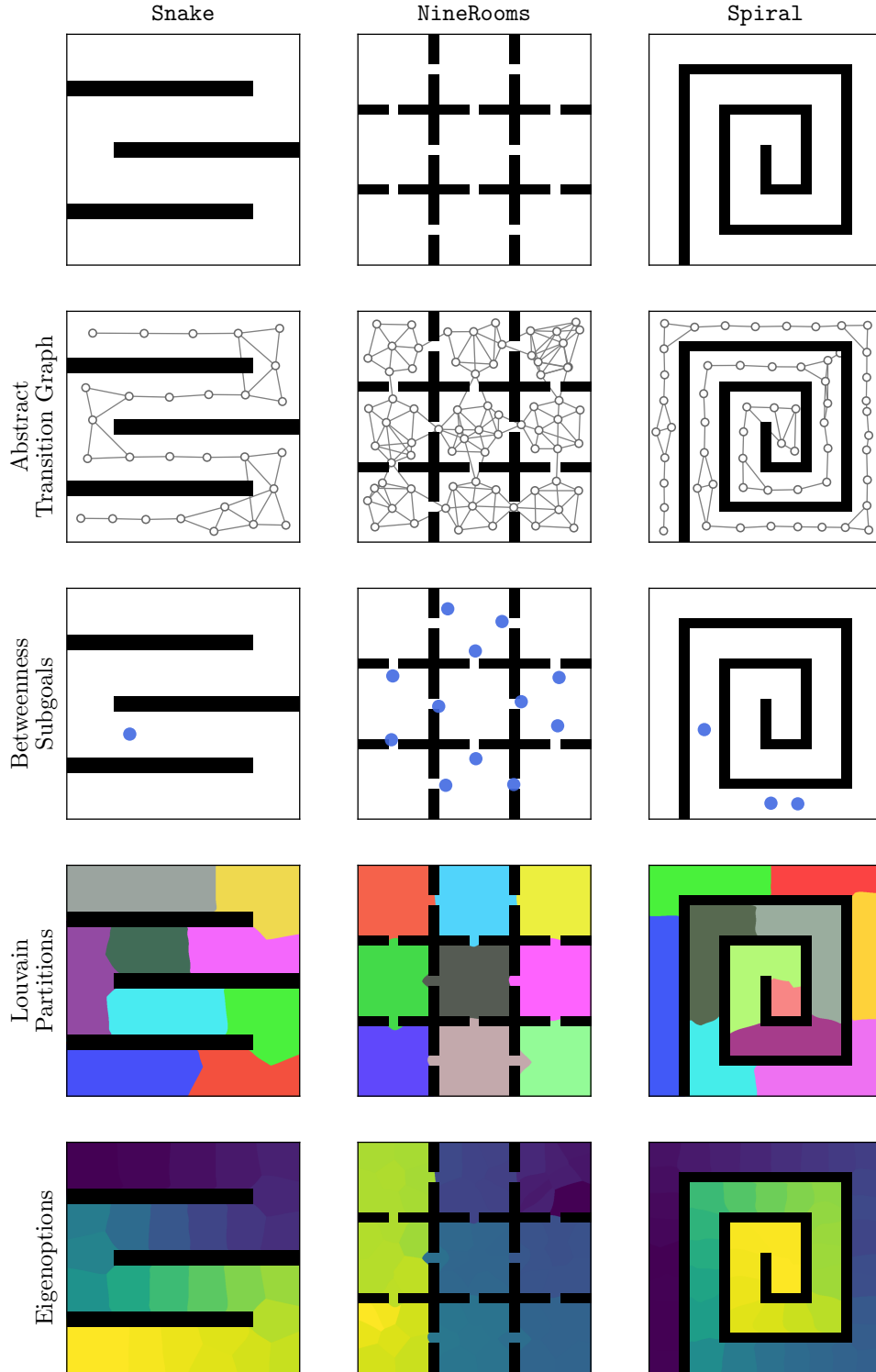


Figure 3: Abstract transition graphs and skills generated using the proposed approach across three environments. Self-loops are omitted from the graphs for clarity. Betweenness denotes the centroids of abstract states identified as subgoals. The Louvain partitions are the highest levels of the hierarchies. Eigenoptions visualises the eigenvector with the second smallest eigenvalue (where brightness corresponds to value).

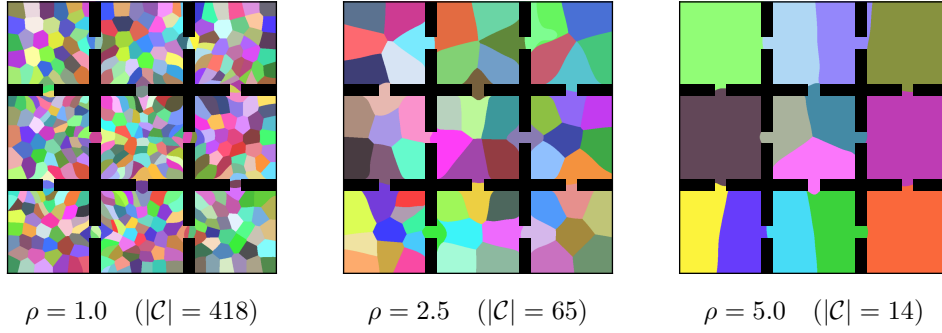


Figure 4: Effect of the BIRCH radius threshold on partitions generated from the NineRooms environment.

are bidirectional; we therefore treat the abstract transition graph as undirected, resulting in a symmetric weight matrix. As in the Louvain approach, edge weights represent connection strength; therefore, we define them as $w(u, v) = \exp(-\|\mu_u - \mu_v\|_2)$ for $(u, v) \in \mathcal{E}$. Figure 3 visualises the eigenvectors corresponding to the second smallest eigenvalues, and Figure 6 shows the first five eigenvectors across all environments.

Results. Figure 3 shows that the proposed approach accurately captures the underlying MDP dynamics. The resulting graph vertices provide a uniform coverage of the state space while strictly respecting environmental constraints and boundaries. The extracted betweenness subgoals align with expectations: in environments lacking strict structural bottlenecks (Snake and Spiral), centrality correctly peaks at the geometric centre. In NineRooms, the approach successfully recovers the doorway bottlenecks. The Louvain partitions in NineRooms successfully capture community structure, separating the state space into the individual rooms. Furthermore, the eigenvectors shown for Snake and Spiral vary smoothly along the most temporally-extended direction of the environments.

As the radius threshold ρ increases, the proposed approach groups together states over progressively longer time horizons. As shown in Figure 4, clusters are well-formed at all three threshold levels. One caveat is that, at low thresholds ($\rho \leq 1.0$), the resulting abstract graphs may be densely connected. Because transitions can jump between disparate clusters due to embedding noise or highly stochastic dynamics, the graph can become highly connected; while this reflects valid transitions, it could hinder downstream skill discovery.

6 Discussion and Future Work

We presented an approach for constructing abstract transition graphs in environments with continuous state spaces by clustering states within a learned temporal distance space. By constructing explicit graphical representations, this approach enables graph-based skill discovery approaches, such as betweenness skills and the Louvain skill hierarchy, to be applied directly to continuous-state domains.

While the proposed approach is conceptually general, the choice of temporal distance metric directly constrains the clustering algorithms that can be applied. We used the Hilbert representation [42] because it embeds states into a Euclidean vector space $\mathbb{R}^d$ where ℓ_2 distance approximates temporal distance, enabling efficient, centroid-based clustering via BIRCH [41]. However, Euclidean distances are inherently symmetric ($d(s, s') = d(s', s)$), whereas environment transition dynamics are frequently asymmetric. While asymmetric alternatives such as quasimetric networks [46], minimum action distance [47], or contrastive successor features [48] accurately capture directed reachability, they do not embed into vector spaces with standard norms. Applying them within the proposed approach would require explicit symmetrisation or dissimilarity-based clustering methods like k -medoids [35], which carry quadratic scaling costs.

The radius threshold ρ provides a natural way to control abstraction granularity. Because distance in the latent space reflects temporal distance in the environment, changing the threshold directly governs

the temporal horizon encompassed by each abstract state. While the Louvain algorithm constructs multi-scale skills via hierarchical graph partitioning on a fixed graph, varying the radius threshold ρ offers an alternative path to multi-scale abstractions. For example, one could construct a sequence of flat transition graphs at varying temporal resolutions, enabling spectral or centrality-based skill discovery across multiple timescales.

Our current evaluation uses an offline dataset of transitions to isolate graph construction from exploration dynamics. However, because BIRCH can be updated incrementally, the proposed approach is well-suited to online deployment. A natural extension is to update the latent distance encoder and the abstract transition graph concurrently alongside agent exploration, dynamically refining the graph as new regions are discovered.

An important next step is to evaluate the discovered skills in downstream reinforcement learning tasks to verify that they accelerate learning and improve sample efficiency in sparse-reward environments. Notably, the Hilbert representation used here supports zero-shot goal-conditioned policies in the latent space, providing a path for generating option policies without further training. Nevertheless, stably combining options with non-linear function approximation remains an open algorithmic challenge that must be addressed to fully leverage these graphical abstractions.

Acknowledgments

We thank the members of the Bath Reinforcement Learning Laboratory for their constructive feedback. Harvey Ayling is funded by the Department of Computer Science at the University of Bath. This work was supported by the Engineering and Physical Sciences Research Council (EPSRC) [grant number EP/X025470/1].

References

- [1] Özgür Şimşek and Andrew G. Barto. Skill characterization based on betweenness. In *Advances in Neural Information Processing Systems*, NeurIPS '08. Curran Associates, Inc., 2008.
- [2] Özgür Şimşek. *Behavioral building blocks for autonomous agents: Description, identification, and learning*. PhD thesis, University of Massachusetts Amherst, 2008.
- [3] Parham Moradi, Mohammad Ebrahim Shiri, and Negin Entezari. Automatic skill acquisition in reinforcement learning agents using connection bridge centrality. In *International Conference on Future Generation Communication and Networking*, pages 51–62. Springer, 2010.
- [4] Parham Moradi, Mohammad Ebrahim Shiri, Ali Ajdari Rad, Alireza Khadivi, and Martin Hasler. Automatic skill acquisition in reinforcement learning using graph centrality measures. *Intelligent Data Analysis*, 16(1):113–135, 2012.
- [5] Joshua B. Evans and Özgür Şimşek. Creating multi-level skill hierarchies in reinforcement learning. In *Advances in Neural Information Processing Systems*, NeurIPS '23. Curran Associates, Inc., 2023.
- [6] Shie Mannor, Ishai Menache, Amit Hoze, and Uri Klein. Dynamic abstraction in reinforcement learning via clustering. In *Proceedings of the 21st International Conference on Machine Learning*, ICML '04. Association for Computing Machinery, 2004.
- [7] Joshua B. Evans. *Identifying multi-level behaviour hierarchies using modularity maximisation*. PhD thesis, University of Bath, 2026.
- [8] Özgür Şimşek and Andrew G. Barto. Using relative novelty to identify useful temporal abstractions in reinforcement learning. In *Proceedings of the 21st International Conference on Machine Learning*, ICML '04. Association for Computing Machinery, 2004.
- [9] Ishai Menache, Shie Mannor, and Nahum Shimkin. Q-cut - dynamic discovery of sub-goals in reinforcement learning. In *Proceedings of the 13th European Conference on Machine Learning*, pages 295—306. Springer-Verlag, 2002.

- [10] Xiao Xu, Mei Yang, and Ge Li. Constructing temporally extended actions through incremental community detection. *Computational Intelligence and Neuroscience*, 2018.
- [11] Marlos C. Machado, Marc G. Bellemare, and Michael Bowling. A Laplacian framework for option discovery in reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, ICML’17. JMLR, 2017.
- [12] Matheus Mendonça, Artur Ziviani, and André Barreto. Laplacian using abstract state transition graphs: A framework for skill acquisition. In *2019 8th Brazilian Conference on Intelligent Systems (BRACIS)*, pages 263–268. IEEE, 2019.
- [13] Amitay Bar, Ronen Talmon, and Ron Meir. Option discovery in the absence of rewards with manifold analysis. In *Proceedings of the 37th International Conference on Machine Learning*, ICML ’20. JMLR, 2020.
- [14] Marlos C. Machado, Clemens Rosenbaum, Xiaoxiao Guo, Miao Liu, Gerald Tesauro, and Murray Campbell. Eigenoption discovery through the deep successor representation. In *Proceedings of the 6th International Conference on Learning Representations*, ICLR ’18, 2018.
- [15] Yifan Wu, George Tucker, and Ofir Nachum. The Laplacian in RL: Learning representations with efficient approximations. In *Proceedings of the 7th International Conference on Learning Representations*, ICLR’19, 2019.
- [16] Martin Klissarov and Marlos C. Machado. Deep Laplacian-based options for temporally-extended exploration. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR, 2023.
- [17] Diego Gomez, Michael Bowling, and Marlos C. Machado. Proper Laplacian representation learning. In *Proceedings of the 12th International Conference on Learning Representations*, ICLR’24, 2024.
- [18] Richard Bellman. A Markovian decision process. *Journal of Mathematics and Mechanics*, 6(5): 679–684, 1957.
- [19] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1): 181–211, 1999.
- [20] Doina Precup. *Temporal abstraction in reinforcement learning*. PhD thesis, University of Massachusetts Amherst, 2000.
- [21] Özgür Şimşek, Alicia P. Wolfe, and Andrew G. Barto. Identifying useful subgoals in reinforcement learning by local graph partitioning. In *Proceedings of the 22nd International Conference on Machine Learning*, ICML ’05. Association for Computing Machinery, 2005.
- [22] Jan Hendrik Metzen. Online skill discovery using graph-based clustering. In *Proceedings of the 10th European Workshop on Reinforcement Learning*, pages 77–88. PMLR, 2013.
- [23] Linton C. Freeman. A set of measures of centrality based on betweenness. *Sociometry*, 40: 35–41, 1977.
- [24] Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001.
- [25] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, (10):P10008, 2008.
- [26] Sridhar Mahadevan and Mauro Maggioni. Proto-value functions: A Laplacian framework for learning representation and control in Markov decision processes. *Journal of Machine Learning Research*, 8(74):2169–2231, 2007.
- [27] Martin Klissarov, Akhil Bagaria, Ziyang Luo, George Konidakis, Doina Precup, and Marlos C. Machado. Discovering temporal structure: An overview of hierarchical reinforcement learning. *arXiv preprint arXiv:2506.14045*, 2025.

- [28] Amnon Attali, Pedro Cisneros-Velarde, Marco Morales, and Nancy M Amato. Discrete state-action abstraction via the successor representation. *arXiv preprint arXiv:2206.03467*, 2022.
- [29] Seungjae Lee, Daesol Cho, Jonghae Park, and H Jin Kim. CQM: Curriculum reinforcement learning with a quantized world model. In *Advances in Neural Information Processing Systems*, NeurIPS '23, 2023.
- [30] Scott Niekum and Andrew G. Barto. Clustering via Dirichlet process mixture models for portable skill discovery. In *Advances in Neural Information Processing Systems*, NeurIPS '11. Curran Associates, Inc., 2011.
- [31] Matheus Mendonça, Artur Ziviani, and André Barreto. Abstract state transition graphs for model-based reinforcement learning. In *2018 7th Brazilian Conference on Intelligent Systems (BRACIS)*, pages 115–120. IEEE, 2018.
- [32] Seungjae Lee, Jigang Kim, Inkyu Jang, and H Jin Kim. DHRL: A graph-based approach for long-horizon and sparse hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems*, NeurIPS '22. Curran Associates, Inc., 2022.
- [33] James MacQueen. Some methods of classification and analysis of multivariate observations. In *Proceedings of Fifth Berkeley Symposium on Mathematical Statistics and Probability*, pages 281–297, 1967.
- [34] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.
- [35] Leonard Kaufman and Peter J. Rousseeuw. Partitioning around medoids (Program PAM). In *Finding Groups in Data: An Introduction to Cluster Analysis*, chapter 2, pages 68–125. John Wiley & Sons, Inc., 1990.
- [36] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*, volume 96, pages 226–231. AAAI Press, 1996.
- [37] Ricardo J. G. B. Campello, Davoud Moulavi, and Jörg Sander. Density-based clustering based on hierarchical density estimates. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 160–172. Springer, 2013.
- [38] Mihael Ankerst, Markus M Breunig, Hans-Peter Kriegel, and Jörg Sander. OPTICS: Ordering points to identify the clustering structure. *ACM SIGMOD Record*, 28(2):49–60, 1999.
- [39] Carl Rasmussen. The infinite Gaussian mixture model. In *Advances in Neural Information Processing Systems*, NeurIPS '99. MIT Press, 1999.
- [40] Fionn Murtagh and Pedro Contreras. Algorithms for hierarchical clustering: An overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 2(1):86–97, 2012.
- [41] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. BIRCH: An efficient data clustering method for very large databases. *ACM SIGMOD Record*, 25(2):103–114, 1996.
- [42] Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with Hilbert representations. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24. JMLR*, 2024.
- [43] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [44] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [45] Diederik P. Kingma and Jimmy Lei Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations, ICLR '15*, 2015.

- [46] Tongzhou Wang, Antonio Torralba, Phillip Isola, and Amy Zhang. Optimal goal-reaching reinforcement learning via quasimetric learning. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR, 2023.
- [47] Lorenzo Steccanella, Joshua B. Evans, Özgür Şimşek, and Anders Jonsson. Learning the minimum action distance. In *Proceedings of the 43rd International Conference on Machine Learning*, ICML '26, 2026.
- [48] Vivek Myers, Chongyi Zheng, Anca Dragan, Sergey Levine, and Benjamin Eysenbach. Learning temporal distances: Contrastive successor features can provide a metric structure for decision-making. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR, 2024.

A Appendix

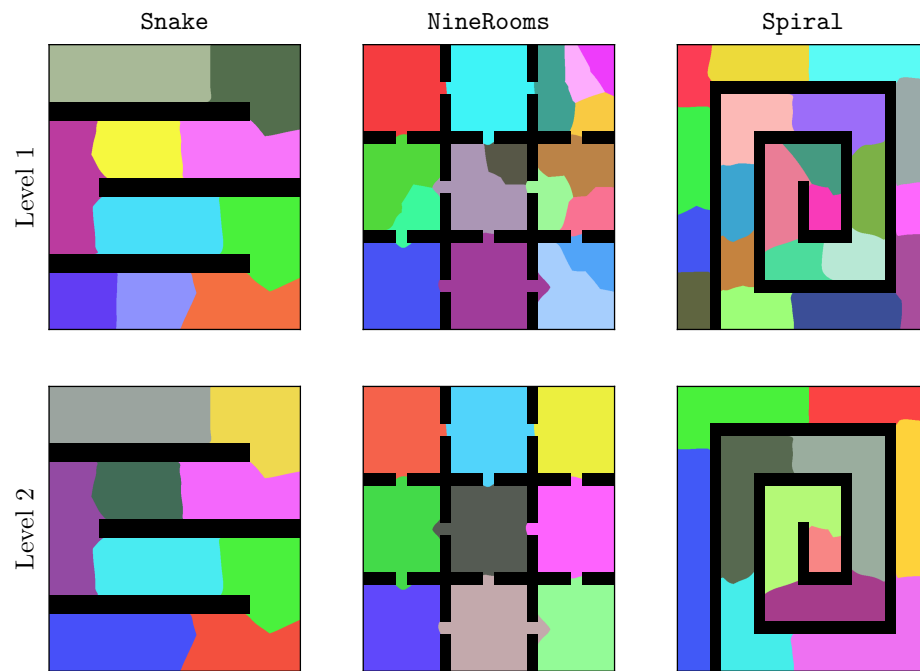


Figure 5: The Louvain skill hierarchy in each environment.

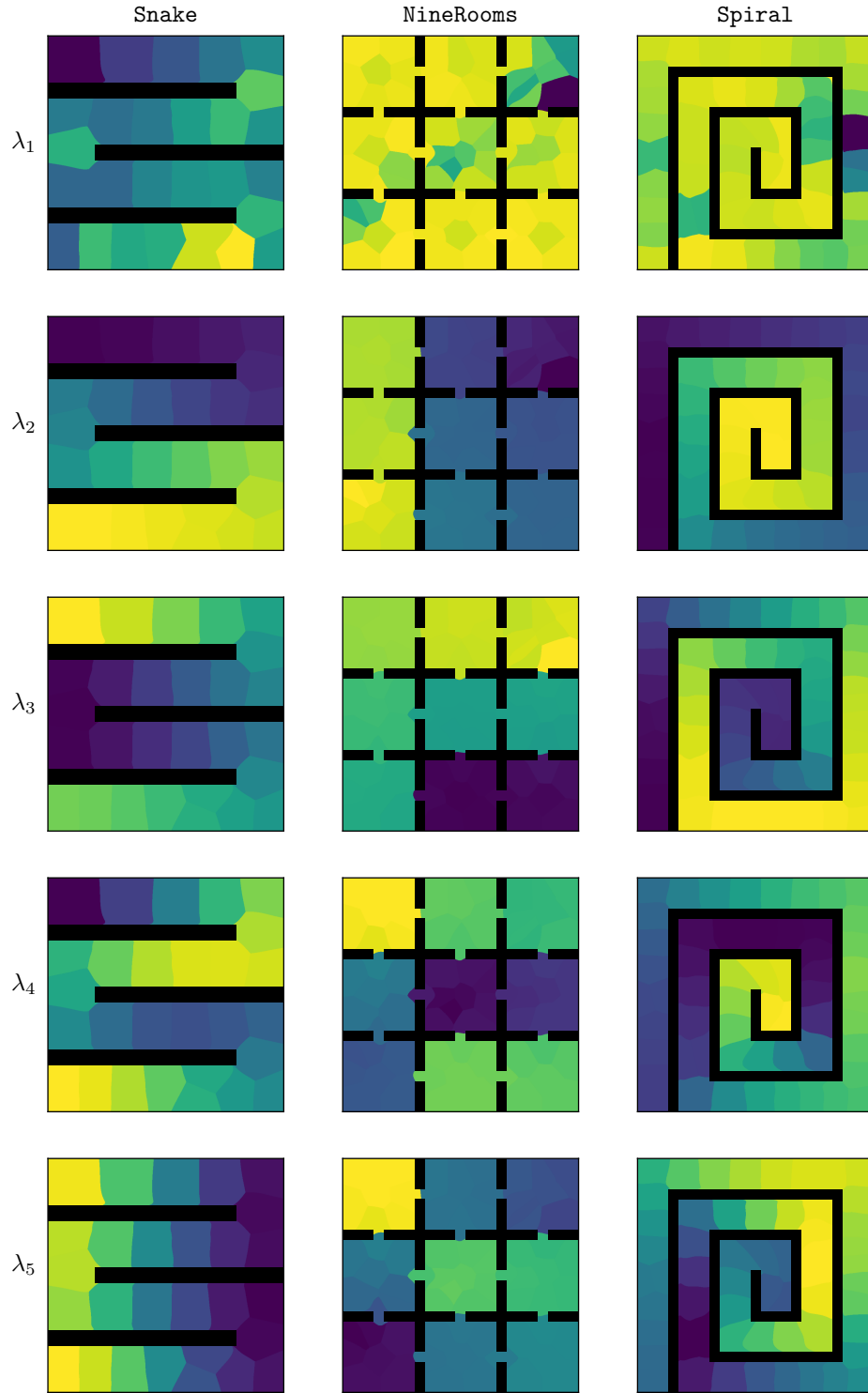


Figure 6: The first five eigenvectors of the normalised graph Laplacian with the smallest eigenvalues. Brightness corresponds to the eigenvector's value for the abstract state.